

A Mathematical Explanation of Transformers

Xue-Cheng Tai*, Hao Liu†, Lingfeng Li‡, Raymond H. Chan§

Abstract

The Transformer architecture has revolutionized the field of sequence modeling and underpins the recent breakthroughs in large language models (LLMs). However, a comprehensive mathematical theory that explains its structure and operations remains elusive. In this work, we propose a novel continuous framework that rigorously interprets the Transformer as a discretization of a structured integro-differential equation. Within this formulation, the self-attention mechanism emerges naturally as a non-local integral operator, and layer normalization is characterized as a projection to a time-dependent constraint. This operator-theoretic and variational perspective offers a unified and interpretable foundation for understanding the architecture’s core components, including attention, feedforward layers, and normalization. Our approach extends beyond previous theoretical analyses by embedding the entire Transformer operation in continuous domains for both token indices and feature dimensions. This leads to a principled and flexible framework that not only deepens on theoretical insight but also offers new directions for architecture design, analysis, and control-based interpretations. This new interpretation provides a step toward bridging the gap between deep learning architectures and continuous mathematical modeling, and contributes a foundational perspective to the ongoing development of interpretable and theoretically grounded neural network models.

Key words: Transformer, attention, operator splitting, integro-differential equation

1 Introduction

Deep neural networks (DNNs) have revolutionized numerous fields, including natural language processing [17, 61], computer vision [28], scientific computing [36, 42, 31], and medical diagnostics [40, 25]. Among these architectures, Transformers [52] have recently emerged as particularly powerful tools, underpinning remarkable successes in large language models (LLMs) such as GPT-3 and GPT-4 [52]. Besides language processing [65], the Transformer and its variants have also been applied in applications, including image processing [10, 47, 41], graph processing [62], operator learning [8, 5, 33, 60].

Recently, a series of works have been conducted to study the theoretical foundation or interpretability of Transformers. Approximation and generalization error of Transformers were studied in [12, 50, 22]. It was shown in [22, 46] that Transformers are adaptive to data’s low-dimensional structures. For interpretability, in [29], it was shown that Transformers are cubic or high-order splines. It was shown in [24] that the vision Transformer can learn spatial structures of the dataset. The Transformer was interpreted as an ODE solver for multi-particle dynamical systems in [11, 13, 38]. The importance of layer normalization in Transformer was studied in

*NORCE Norwegian Research Centre, Nygårdstangen, NO-5838 Bergen, Norway Email: xtai@norceresearch.no, xuechengtai@gmail.com.

†Department of Mathematics, Hong Kong Baptist University, Kowloon Tong, Hong Kong. Email: haoliu@hkbu.edu.hk.

‡Hetao Institute of Mathematics and Interdisciplinary Sciences (Shenzhen). Email: lilingfeng@himis-sz.cn.

§School of Data Science and Department of Operations and Risk Management, Lingnan University, Tuen Mun, Hong Kong. Email: raymond.chan@ln.edu.hk.

[26]. A general introduction to Transformers with mathematical descriptions can be found in [51].

In this work, we will develop a mathematical theoretical framework to explain the Transformer architecture of [52], which has achieved remarkable success in large language models (LLMs) and other sequence modeling tasks. We show that the Transformer can be interpreted as a discretization of the following continuous integro-differential equation:

$$\left\{ \begin{array}{l} u_t = \underbrace{\left\langle \gamma(\mathbf{x}, \cdot, t; u), V(\cdot, \mathbf{y}, t; u) \right\rangle_{\Omega_x}}_{\text{I: attention}} + \underbrace{\partial I_{S_1(\sigma_1(t), \sigma_2(t))}(u)}_{\text{II: layer normalization}} \\ \quad + \underbrace{\sum_{j=1}^J \left(\left\langle W_j(\cdot, \mathbf{y}, t) u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y} + b_j(\mathbf{x}, t) \right)}_{\text{III: fully connected network}} + \partial I_{S_2}(u) \text{ for } t \in (0, T], \\ u(\mathbf{x}, \mathbf{y}, 0) = f(\mathbf{x}, \mathbf{y}), \end{array} \right. \quad (1)$$

for $(\mathbf{x}, \mathbf{y}) \in \Omega_x \times \Omega_y$, where $\mathbf{x}$ denotes the token index, $\mathbf{y}$ the entry of token vectors, Ω_x and Ω_y are continuous domains, and $T > 0$ is a fixed terminal time. This formulation provides a rigorous mathematical interpretation of the Transformer and its key components, such as self-attention and feedforward layers. The definitions of attention, layer normalization, and the fully connected layer, along with detailed explanations of the domains Ω_x , Ω_y , and the control variables, are given in Section 2. Equation (1) highlights the non-local nature of attention and reveals the Transformer as a structured operator derived from a discretized variational principle. We note that several previous works have also established connections between Transformer-based models and multi-particle dynamical systems [11, 13, 38]. These studies interpret the original or modified Transformer models as discretizations of interacting particle systems in a time-continuous setting, treating interactions across spatial variables as particle interactions. In contrast, our formulation (1) is fundamentally different: we show that the Transformer arises as a discretization of a continuous integro-differential equation, offering a more unified operator-theoretic and variational perspective.

This perspective offers several advantages. First, it provides a unifying mathematical framework that connects diverse architectures, such as CNNs, UNets, and Transformers, under the common lens of differential and integral equations. This unification improves our understanding of the design principles underlying modern deep networks and provides insights for cross-architectural studies.

Second, by casting neural network architectures as time-stepping schemes of dynamical systems, our framework allows the systematic exploration of new architectures using well-established tools from numerical analysis. For instance, stability, convergence, and approximation properties of the underlying continuous models can inform the selection of network structures and hyperparameters, leading to more robust and interpretable models.

Third, this approach creates a principled pathway for embedding domain-specific knowledge, such as physical laws, geometric structures, or conservation principles, directly into the design of neural architectures, leading to architectures tailored for specific scientific or engineering tasks.

In summary, our formulation not only advances the theoretical understanding of deep neural networks but also provides actionable tools for the principled design of next-generation models. It bridges the gap between continuous mathematical modeling and discrete network implementation, establishing a foundation for informed, explainable, and application-aware neural network development.

This paper is organized as follows: We introduce the integral equation for Transformer with single-head attention in continuous setting in Section 2. An operator-splitting based algorithm is proposed in Section 3 to solve the integral equation. We show in Section 4 that the discretized algorithm exactly recovers the Transformer encoder in [52], and also discuss how to adapt the

proposed algorithm to recover the architecture of Vision Transformer (ViT) [10]. In Section 5, we extend the integral equation and proposed algorithm to Transformer with multi-head attention. Extension to convolutional Transformer and its relation to Convolutional vision Transformer (CvT) [56] are discussed in Section 6. This paper is concluded in Section 7.

In the rest of this paper, we use bold lowercase letters to denote vectors, normal letters to denote scalars and functions, and calligraphic letters to denote operators.

Related works

It is essential to examine the results of relevant literature and situate our approach within a historical context. Compared to Transformers, more studies can be found for deep neural networks (DNNs). A widely accepted interpretation views DNN as nonlinear high-dimensional function approximators [59, 35, 64, 6]. Under this view, the layers of a DNN perform successive nonlinear transformations that approximate complex mappings from input to output data. This approximation framework has been instrumental in explaining the expressive power of DNNs. Another promising direction for explaining and designing neural network architectures involves interpreting them as discretizations of continuous-time dynamical systems governed by differential or integral equations [7, 55, 4, 54]. E et al. show that DNNs can be viewed as continuous dynamical systems in [54] and special discretization of continuous problems in [55]. In [7], Chen et al. pointed out connections between residual networks and ordinary differential equations, and proposed Neural ODE. Benning et al. investigated the relations between DNNs and optimal control problems [4]. In this class of approach, neural network training is seen as an optimal control problem constrained by a continuous evolution equation. Related to the approach mentioned above, we especially want to mention some recent studies that have proposed a general framework for designing neural network architectures inspired by continuous-time dynamical systems [48, 32]. These methods leverage operator-splitting techniques to construct neural networks, which are so-called continuous neural networks, as discretizations of continuous differential equations. The continuous learning problem is an optimal control problem with this “neural network” as a constraint. In this framework, the evolution of hidden states in a network is governed by differential equations (continuous neural network). The goal of this continuous learning problem is to find optimal controls—represented by learnable parameters—within a discretized dynamical process.

The use of operator-splitting allows these complex dynamics to be decomposed into simpler substeps, each of which corresponds to a specific type of operation (e.g., convolution, nonlinearity). These substeps are then unrolled into individual layers of a neural network. This interpretation not only clarifies the structure and purpose of different architectural components but also facilitates the incorporation of prior knowledge about the underlying data dynamics or physics.

Building on this theory, we have recently provided a rigorous mathematical explanation of the well-known UNet architecture [49]. We demonstrated that UNet can be viewed as a specific discretization of the following simple differential equation, where each encoder and decoder stage corresponds to a particular substep in the operator-splitting scheme:

$$\begin{cases} \frac{\partial u(\mathbf{x}, t)}{\partial t} = W(\mathbf{x}, t) * u(\mathbf{x}, t) + d(t) - \ln \frac{u(\mathbf{x}, t)}{1 - u(\mathbf{x}, t)} + \partial I_{\Sigma}(u), & (\mathbf{x}, t) \in \Omega \times (0, T], \\ u(\mathbf{x}, 0) = f(\mathbf{x}), & \mathbf{x} \in \Omega, \end{cases} \quad (2)$$

where $*$ denotes convolution, I_{Σ} is the indicator function of $\Sigma = \{u : u(\mathbf{x}) \geq 0 \text{ for } \mathbf{x} \in \Omega\}$, f is a function defined in a domain Ω and T is the final time chosen by the user for time variable t . The learnable variables $\theta = \{W(\mathbf{x}, t), d(t)\}$ are regarded as control variables which will be learned in the training process. The mapping $\mathcal{N}_{\theta}(f) \mapsto u(\mathbf{x}, T)$ turns out to be the UNet in the continuous setting. This perspective not only elucidates the structure of the UNet architecture but also provides a foundation for systematic modifications and enhancements. A range of alternative interpretations and mathematically inspired designs of deep neural networks

can be found in the literature, including [45, 4, 20, 44, 19, 23, 30, 34, 18, 58, 2]. For example, neural networks have been interpreted as optimal control problems in [4, 44]. Gregor and LeCun [18] were among the first to connect deep learning with sparse coding through learned iterative shrinkage algorithms. Ruthotto and Haber [45] proposed neural networks derived from partial differential equations (PDEs), while He and Xu [23] introduced MgNet, which employs multigrid principles for feature extraction. Hagemann et al. [21] formulated stochastic normalizing flows as Markov chains to tackle inverse problems. Altekruiger et al. [1] analyzed the robustness of conditional generative model. Long et al. [34] built PDENet based on finite difference schemes to recover PDE models from data. Yang et al. [58] constructed ADMM-CSNet by unrolling the ADMM optimization algorithm into a network architecture. Wang et al. [53] developed a diffusion-residual network inspired by convection-diffusion ODEs for classification tasks. Most recently, Martin et al. [39] introduced a plug-and-play architecture using flow matching for image restoration. These pioneering efforts have provided essential insights and have directly inspired our recent work [48, 49] and the present study.

2 The Continuous Model

In the following, we explain the Transformer in the seminal work in [52] in the continuous setting and show later that proper discretizations of the proposed model will recover the Transformer exactly.

2.1 A brief recap of the discrete Transformer encoder

The Transformer architecture introduced in [52] is built around a core encoder block that processes sequential data through three principal components: a self-attention mechanism, layer normalization, and a feedforward network. In this section, we briefly recall its mathematical formulation, which will serve as the starting point for our continuous interpretation.

Let the input to a Transformer encoder layer be represented by a matrix

$$\mathbf{u} = [\mathbf{u}_1^\top, \mathbf{u}_2^\top, \dots, \mathbf{u}_{N_x}^\top]^\top \in \mathbb{R}^{N_x \times N_y},$$

where N_x is the number of tokens and N_y is the embedding dimension. Each row $\mathbf{u}_i \in \mathbb{R}^{N_y}$ corresponds to the embedded representation of a token.

Self-Attention: The self-attention mechanism transforms $\mathbf{u}$ through learned weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{N_y \times N_y}$ to obtain query, key, and value matrices:

$$\mathbf{Q} = \mathbf{u}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{u}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{u}\mathbf{W}^V.$$

The scaled dot-product attention is then computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{N_y}}\right) \mathbf{V},$$

where the softmax is applied row-wise. This operation allows each token to aggregate information from all tokens in the sequence, weighted by their compatibility.

Layer Normalization: Following [3], layer normalization is applied per token (row-wise). Given an input vector $\mathbf{z} \in \mathbb{R}^{N_y}$, the normalized output is:

$$\text{LayerNorm}(\mathbf{z}) = \frac{\mathbf{z} - \mu}{\sigma} \odot \boldsymbol{\gamma} + \boldsymbol{\beta},$$

where μ and σ are the mean and standard deviation of $\mathbf{z}$, $\odot$ represents the elementwise product and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{N_y}$ are learnable parameters.

Feedforward Network: The feedforward network consists of two linear transformations with a ReLU activation in between:

$$\text{FFN}(\mathbf{z}) = \text{ReLU}(\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2,$$

where $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{N_y \times N_y}$ and $\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^{N_y}$.

Overall Encoder Block: A standard Transformer encoder layer combines these components with residual connections [52]:

$$\begin{aligned} \mathbf{u}' &= \mathbf{u} + \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}), \\ \mathbf{u}'' &= \text{LayerNorm}(\mathbf{u}'), \\ \mathbf{u}''' &= \mathbf{u}'' + \text{FFN}(\mathbf{u}'''), \\ \mathbf{u}_{\text{out}} &= \text{LayerNorm}(\mathbf{u}'''). \end{aligned}$$

This discrete, layer-wise processing will be our reference structure. In what follows, we show that this architecture can be viewed as a numerical discretization of a continuous dynamical system.

2.2 The continuous setting for Transformers

The interpretation of deep neural networks as discretizations of continuous-time dynamical systems has gained considerable traction in recent years [7, 45, 48, 54]. These continuous viewpoints not only deepen theoretical understanding but also provide principled frameworks for architecture design and analysis.

In this section, we propose a control problem in continuous setting, named continuous Transformer, that reproduces the standard Transformer encoder through a structured discretization scheme. The continuous Transformer is a time-dependent integral equation. We will subsequently show that after discretizations by operator splitting for the time variable t and spatial discretizations for the $\mathbf{x}, \mathbf{y}$ variables, the resulting splitting method recovers the Transformer architecture. The Transformer proposed in [52] consists of three components: attention, layer normalization, and feedforward networks. In the derivation of our control problem, we show that each component has a corresponding operation in the integral equation. We first focus on single-head attention. Our framework can be easily extended to the multi-head case, which will be detailed in Section 5.

To make the explanation clearer and simpler, our notations and setup will follow [52], but in a continuous counterpart. We consider bounded sets $\Omega_x = [0, L_x]^{d_x}, \Omega_y = [0, L_y]^{d_y}$ for $L_x, L_y > 0$ and positive integers d_x, d_y . When $d_x = d_y = 1$, this corresponds to a sequence of bounded tokens and a bounded embedding in practice. Let $u(\mathbf{x}, \mathbf{y}, t)$ be a function of $\mathbf{x}, \mathbf{y}$ and time t . In applications of LLMs, $\mathbf{x} \in \Omega_x$ refers to the index of tokens, $\mathbf{y} \in \Omega_y$ refers to entries of token vectors. Please refer to Appendix A for explanations of the physical meanings of $\mathbf{x}, \mathbf{y}$, and u for applications of Transformers to LLM.

As stated in [52], attention is all you need for the Transformer. In fact, we show below that the attention layer has a very simple mathematical explanation in the continuous setting. It consists of integral transformations on the input functions to extract features with three different kernels. We use features extracted from the first two kernels to generate the attention score through a softmax operator, and then multiply the attention score by the feature extracted from the third kernel to compute the output. All three kernels in the integral transformation will be parameterized as learnable variables, see (13).

2.3 Integral Transformations for Attention and Feature Extraction

Let the functions $W^Q(\mathbf{y}, \tilde{\mathbf{y}}, t)$, $W^K(\mathbf{y}, \tilde{\mathbf{y}}, t)$, and $W^V(\mathbf{y}, \tilde{\mathbf{y}}, t)$ be three kernels defined on $\Omega_y \times \Omega_y \times [0, T]$, and they will be learned from data as shown in LLM applications. See Appendix A

for explanations of the physical meanings of $\mathbf{x}, \mathbf{y}, u(\mathbf{x}, \mathbf{y}, t)$. For a given function $u(\mathbf{x}, \mathbf{y}, t)$, we define integral transformations:

$$Q(\mathbf{x}, \mathbf{y}, t; u) = \left\langle W^Q(\cdot, \mathbf{y}, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y} = \int_{\Omega_y} W^Q(\xi, \mathbf{y}, t) u(\mathbf{x}, \xi, t) d\xi, \quad (3)$$

$$K(\mathbf{x}, \mathbf{y}, t; u) = \left\langle W^K(\cdot, \mathbf{y}, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y} = \int_{\Omega_y} W^K(\xi, \mathbf{y}, t) u(\mathbf{x}, \xi, t) d\xi, \quad (4)$$

$$V(\mathbf{x}, \mathbf{y}, t; u) = \left\langle W^V(\cdot, \mathbf{y}, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y} = \int_{\Omega_y} W^V(\xi, \mathbf{y}, t) u(\mathbf{x}, \xi, t) d\xi. \quad (5)$$

Here and later, we use $\langle \cdot, \cdot \rangle_{\Omega_y}$ to denote L^2 inner product on a given domain Ω_y . Its induced norm will be denoted by $\|\cdot\|_{\Omega_y}$. Functions Q and K will be used to generate attention scores, and V is used to extract features from u . When all these functions are discretized over a pixel-type grid over $\Omega_x \times \Omega_y$, the discrete counterparts of these functions are represented by matrices and the discrete counterparts of the integral transformations are just standard matrix multiplications, see Section 3.4, especially formula (35). We compute the attention score by

$$\gamma(\mathbf{x}, \tilde{\mathbf{x}}, t; u) = \text{Softmax}_2 \left(\frac{1}{\sqrt{|\Omega_y|}} \left\langle Q(\mathbf{x}, \cdot, t; u), K(\tilde{\mathbf{x}}, \cdot, t; u) \right\rangle_{\Omega_y} \right), \quad (6)$$

where $\gamma(\mathbf{x}, \tilde{\mathbf{x}}, t)$ is defined on $\Omega_x \times \Omega_x \times [0, T]$, Softmax_2 represents the softmax function along the second dimension defined as:

$$\text{Softmax}_2(a(\mathbf{x}, \tilde{\mathbf{x}}, t)) = \frac{\exp(a(\mathbf{x}, \tilde{\mathbf{x}}, t))}{\int_{\Omega_x} \exp(a(\mathbf{x}, \eta, t)) d\eta}.$$

The output of the attention layer is then obtained by taking the inner product of the attention score γ and the feature V : $\left\langle \gamma(\mathbf{x}, \cdot, t; u), V(\cdot, \mathbf{y}, t; u) \right\rangle_{\Omega_x}$. We emphasize that the first two integral transformations in (3)–(4) are used to generate the attention score $\gamma(\mathbf{x}, \tilde{\mathbf{x}}, t; u)$ and the last integral transformation (5) will be used for feature extractions. All three integral kernels will be learned during training, see explanations later in this section.

2.4 Mathematical formulations for the Layer Normalization

The layer normalization [3] in the Transformer [52] and also many other neural networks [66, 63, 57, 9] can be mathematically described as a projection of a function to a set with given mean value σ_1 and variance σ_2^2 . Define sets

$$S_1(\sigma_1, \sigma_2) = \left\{ u : \frac{1}{|\Omega_y|} \int_{\Omega_y} u(\mathbf{x}, \xi, t) d\xi = \sigma_1, \right. \\ \left. \frac{1}{|\Omega_y|} \int_{\Omega_y} (u(\mathbf{x}, \xi, t) - \sigma_1)^2 d\xi = \sigma_2^2 \right\}, \quad (7)$$

$$S_2 = \{u : u \geq 0\}, \quad (8)$$

and their corresponding indicator functions

$$I_{S_1(\sigma_1, \sigma_2)}(u) = \begin{cases} 0 & \text{if } u \in S_1, \\ +\infty & \text{otherwise,} \end{cases} \quad I_{S_2}(u) = \begin{cases} 0 & \text{if } u \in S_2, \\ +\infty & \text{otherwise.} \end{cases} \quad (9)$$

Later, we shall show that layer normalization is just a projection of a function into the set $S_1(\sigma_1, \sigma_2)$, see Section 3.3.2 and the ReLU activation function is just a projection of a function to the set S_2 , see Section 3.3.3.

2.5 Our continuous Transformer

In addition to the control variables $W^Q(\mathbf{y}, \tilde{\mathbf{y}}, t)$, $W^K(\mathbf{y}, \tilde{\mathbf{y}}, t)$, $W^V(\mathbf{y}, \tilde{\mathbf{y}}, t)$, let us also introduce control variables $\{W_j(\tilde{\mathbf{y}}, \mathbf{y}, t)\}_{j=1}^J$ on $\Omega_y \times \Omega_y \times [0, T]$, and $b(\mathbf{x}, t)$, $b_j(\mathbf{x}, t)$ on $\Omega_x \times [0, T]$. We call the following continuous integral equation a **Continuous Transformer**:

$$\begin{cases} u_t = \underbrace{\left\langle \gamma(\mathbf{x}, \cdot, t; u), V(\cdot, \mathbf{y}, t; u) \right\rangle_{\Omega_x}}_{\text{I: attention}} + \underbrace{\frac{\partial I_{S_1(\sigma_1(t), \sigma_2(t))}(u)}{\partial t}}_{\text{II: layer normalization}} \\ \quad + \underbrace{\sum_{j=1}^J \left(\left\langle W_j(\cdot, \mathbf{y}, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y} + b_j(\mathbf{x}, t) \right)}_{\text{III: fully connected network}} + \frac{\partial I_{S_2}(u)}{\partial t}, \quad t \in (0, T], \\ u(\mathbf{x}, \mathbf{y}, 0) = f(\mathbf{x}, \mathbf{y}). \end{cases} \quad (10)$$

For simplicity and to align with common practice in Transformer implementations, we equip (10) with zero boundary conditions or periodic boundary conditions in both $\mathbf{x}$ and $\mathbf{y}$. This is commonly used in LLMs and image processing applications.

We shall show in the rest of this paper that the Transformer proposed in [52] is just a discretization of this continuous Transformer. In (10), f is some initial state, T is a fixed time chosen by the user. Following conventions in the literature, we denote all the control variables as

$$\theta = \{W^Q(\mathbf{x}, \mathbf{y}, t), W^K(\mathbf{x}, \mathbf{y}, t), W^V(\mathbf{x}, \mathbf{y}, t), \{W_j(\mathbf{x}, \mathbf{y}, t), b_j(\mathbf{x}, t)\}_{j=1}^J, \sigma_1(t), \sigma_2(t)\}. \quad (11)$$

The Continuous Transformer is the mapping:

$$\mathcal{N}_\theta : f \mapsto u(\mathbf{x}, \mathbf{y}, T). \quad (12)$$

In Section 3, we will discretize (10) by the operator-splitting method for the time variable and use a uniform grid for the spatial variables $\mathbf{x}, \mathbf{y}$. After discretization, the continuous Transformer becomes the Transformer proposed in [52]. In relation to this equivalence, the operations in the right-hand side of (10) can be classified into three sets:

- Operations in Set I correspond to attentions in Transformers. It computes the attention score $\gamma(\mathbf{x}, \tilde{\mathbf{x}}, t; u)$ and applies it to the extracted features in $V(\mathbf{x}, \mathbf{y}, t)$.
- Operations in Set II correspond to layer normalization. For each token location $\mathbf{x}$, it requires function u to have mean σ_1 and variance σ_2^2 along the $\mathbf{y}$ direction. In other words, for each token, the corresponding series is expected to have mean σ_1 and variance σ_2^2 .
- Operations in Set III correspond to feedforward networks with J layers. The first component corresponds to linear layers, each of which contains one linear transformation W_j and a bias b_j . The second component corresponds to activation functions. This term projects u to S_1 , which can be realized by ReLU functions.

2.6 The continuous control problem for learning

Our objective is to optimize θ so that given an input $f(\mathbf{x}, \mathbf{y})$, the control problem will drive $u(\mathbf{x}, \mathbf{y}, T)$ to a desired state v . Specifically, given a dataset $\{(u_i, v_i)\}_{i=1}^B$, where u_i is the input and v_i is the target state, let $\ell(\cdot, \cdot)$ be a loss function measuring the discrepancy between its arguments. Let $\theta, \mathcal{N}_\theta$ be as defined in (11)–(12), the training process with the loss function ℓ is in fact a process to optimize θ for the following minimization problem:

$$\min_{\theta} \frac{1}{B} \sum_{i=1}^B \ell(\mathcal{N}_\theta(u_i), v_i). \quad (13)$$

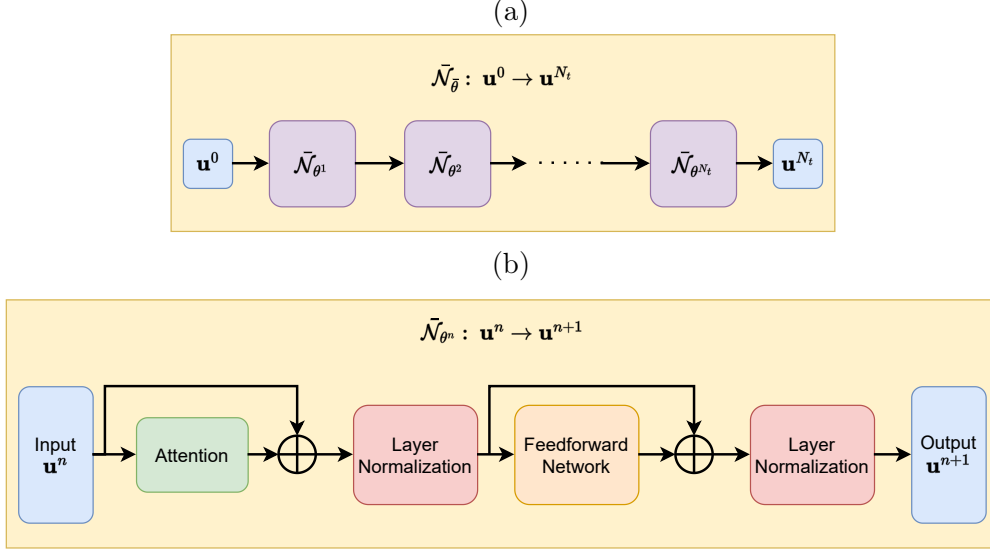


Figure 1: (a) Illustration of the whole evolution. (b) Illustration of the discretized operator-splitting scheme (16)–(21).

This is essentially an optimization problem constrained by an integral differential equation of the following form:

$$\min_{\theta} \frac{1}{B} \sum_{i=1}^B \ell(w_i, v_i), \text{ under constraint } w_i = \mathcal{N}_{\theta}(u_i) \text{ solves the control problem (10).} \quad (14)$$

3 Discretizations of the Continuous Problem

Following the idea of PottsMGNet [48], in this section, we use some proper operator-splitting methods to discretize the time variable for (10). For a review about operator splitting methods, please refer to [14, 15, 16]. Specifically, we discretize the temporal domain of (10) by sequential splitting, c.f. [16, Sec. 2.2]. The number of sequential splittings corresponds to the number of layers in Transformers. Spatial discretization for $\mathbf{x}$ and $\mathbf{y}$ will be discussed in Section 3.4. The discretization of $\mathbf{x}$ determines the number of tokens in the input, and the discretization of $\mathbf{y}$ determines the size of the embedding vector for each token.

3.1 The overall time discretization

Let $\{t^n\}_{n=0}^{N_t}$ be the set of time grids with time step $\Delta t = T/N_t$. We denote the numerical solution of (13) at time t^n by u^n . We will design a numerical scheme to approximate $\mathcal{N}_{\theta}$. Specifically, for each t^n , we design a discrete propagator $\tilde{\mathcal{N}}^n$ propagating u^{n-1} to u^n . Then, problem (10) is numerically approximated by

$$u(t^{N_t}) \approx u^{N_t} = \tilde{\mathcal{N}}^{N_t} \circ \tilde{\mathcal{N}}^{N_t-1} \circ \dots \circ \tilde{\mathcal{N}}^1(f),$$

see Figure 1(a) for an illustration.

In each $\tilde{\mathcal{N}}^n$, time-discretized control variables θ will be used. We denote the control variables used in $\tilde{\mathcal{N}}^n$ by θ^n . To show the dependence of $\tilde{\mathcal{N}}^n$ on the control variable θ^n , we will also use $\tilde{\mathcal{N}}_{\theta^n}^n$ to denote the propagator $\tilde{\mathcal{N}}^n$. From the following subsections, c.f. §3.4, we will see that

$$\theta^n = \{W^{Q,n-1}, W^{K,n-1}, W^{V,n-1}, \{W_j^{n-1}, b_j^{n-1}\}_{j=1}^J, \sigma_1^{n-1}, \sigma_2^{n-1}\}.$$

Denote all of the discretized variables by $\bar{\theta}$, i.e. $\bar{\theta} = \{\theta^n\}_{n=1}^{N_t}$. The whole evolution procedure from f to u^{N_t} by $\bar{\mathcal{N}}_{\bar{\theta}}$, i.e., $\bar{\mathcal{N}}_{\bar{\theta}} = \bar{\mathcal{N}}_{\theta^{N_t}}^{N_t} \circ \bar{\mathcal{N}}_{\theta^{N_t-1}}^{N_t-1} \circ \dots \circ \bar{\mathcal{N}}_{\theta^1}^1$. Given a dataset $\{(u_i, v_i)\}_{i=1}^B$, then the discrete training problem is to solve the following constrained optimization problem:

$$\min_{\bar{\theta}} \frac{1}{B} \sum_{i=1}^B \ell(\bar{\mathcal{N}}_{\bar{\theta}}(u_i), v_i). \quad (15)$$

3.2 Time Discretizations by Operator Splitting Method

In the following sections, we illustrate our numerical scheme for propagator $\bar{\mathcal{N}}_{\theta^1}^1$. Propagator $\bar{\mathcal{N}}_{\theta^n}^n$ for $n = 2, \dots, N_t$ is exactly the same by replacing θ^1 by θ^n . Our numerical scheme is designed based on operator splitting methods. For simplicity, we take $\Delta t = 1$. It is often used in data analysis and image processing literature and is equivalent to changing the network parameters properly. In the following, with a slight abuse of notation, we denote the time-discretized version of any time-dependent function $a(\cdot, t)$ by $a^n(\cdot) := a(\cdot, t^n)$, where t^n is the n -th time step. Using the Lie scheme [16, Sec. 2.2] as explained in Appendix C, given $u^0 = f$, we compute u^1 by $M = 4 + J$ substeps:

Substep 1: Solve $u^{1/M}$ from

$$u^{1/M} - u^0 = \left\langle \gamma^0(\mathbf{x}, \cdot; u^0), V^0(\cdot, \mathbf{y}; u^0) \right\rangle_{\Omega_x}. \quad (16)$$

Substep 2: Compute $u^{2/M}$ from

$$u^{2/M} - u^{1/M} = \partial I_{S_1(\sigma_1^0, \sigma_2^0)}(u^{1/M}). \quad (17)$$

Substep 2+j : Compute $u^{(2+j)/M}$ sequentially for $j = 1, \dots, J-1$ from

$$u^{(2+j)/M} - u^{(1+j)/M} = \left\langle W_j^0(\cdot, \mathbf{y}), u^{(1+j)/M}(\mathbf{x}, \cdot) \right\rangle_{\Omega_y} + b_j^0(\mathbf{x}) + \partial I_{S_2}(u^{(2+j)/M}). \quad (18)$$

Substep 2+J : Compute $u^{(2+J)/M}$ from

$$u^{(2+J)/M} - u^{(1+J)/M} = \left\langle W_J^0(\cdot, \mathbf{y}), u^{(1+J)/M}(\mathbf{x}, \cdot) \right\rangle_{\Omega_y} + b_J^0(\mathbf{x}). \quad (19)$$

Substep 3+J: Compute $u^{(3+J)/M}$ from

$$u^{(3+J)/M} = \frac{1}{2}(u^{(2+J)/M} + u^{2/M}). \quad (20)$$

Substep 4+J: Compute u^1 from

$$u^1 - u^{(3+J)/M} = \partial I_{S_1(\sigma_1^0, \sigma_2^0)}(u^1). \quad (21)$$

Note that in (16)–(19) and (21), the left hand side $u^{(k+1)/M} - u^{k/M} = \frac{u^{(k+1)/M} - u^{k/M}}{\Delta t}$ with $\Delta t = 1$ is the finite difference approximation of u_t .

Substep 1 to Substep (4 + J) evolve u^0 to u^1 by sequentially passing it through all operators in (10), corresponding to all components in Transformer from input to output. Substep 1 corresponds to the attention layer and the skip connection. Substep 2 corresponds to layer normalization. Substep 3 to Substep (2 + J) correspond to the J layers in the fully connected network. Substep 3 + J corresponds to the skip connection after the fully connected network. The last substep corresponds to the final layer normalization. From here, we can already see that each substep in the Lie splitting scheme corresponds to one layer in the Transformer. In

[52], it used $J = 2$ and thus $M = 6$. The different layers in the Transformer is just a sequential computation of the substep solutions:

$$\begin{aligned} u^0 \quad (\text{input token}) &\mapsto u^{1/6}(\text{attention}) \mapsto u^{2/6}(\text{normalization layer}) \\ &\mapsto u^{3/6} \text{ and } u^{4/6}(\text{fully connected layer with ReLU}) \\ &\mapsto u^{5/6}(\text{skip connection layer}) \mapsto u^1(\text{normalization layer}). \end{aligned} \quad (22)$$

Remark 1. In scheme (16)–(21), substep (20) is a relaxation step and substep (17) and (21) repeat $\partial I_{S_1(\sigma_1^0, \sigma_2^0)}$ twice. These two are numerical techniques commonly used to improve algorithm stability and accuracy. For example, it is shown in [27] that applying relaxation technique (averaging step) to Runge-Kutta methods helps guarantee conservation or stability with respect to any inner-product norm. For another example, compared to Lie’s splitting, the Strang splitting intentionally repeat an operator twice which improves the accuracy from first order to second order [15].

Remark 2. The Lie splitting scheme was used for three reasons: (1) The layer-by-layer structure of the Transformer naturally aligns with a sequential splitting approach, where each operator (attention, normalization, feedforward) is applied in a stepwise manner, mirroring the sequential flow of data through the encoder block. (2) Sequential splitting allows each substep to be mapped directly to a specific component of the Transformer. This enhances interpretability and facilitates the extension of the framework to variants such as Vision Transformers and Convolutional Transformers. (3) While parallel splitting could also be applied, they do not directly recover the standard Transformer’s serial layer composition. The sequential scheme is thus the most natural choice for reproducing the Transformer architecture.

3.3 Solutions to Each Subproblem

In the following, we supply the details for the solution to each subproblem in the splitting scheme (16)–(21).

3.3.1 Solution to Subproblem (16)

We have

$$u^{1/M} = u^0 + \int_{\Omega_x} \text{Softmax}_2 \left(\frac{1}{\sqrt{|\Omega_y|}} \left\langle Q^0(\mathbf{x}, \cdot; u^0), K^0(\eta, \cdot; u^0; \cdot) \right\rangle_{\Omega_y} \right) V^0(\eta, \mathbf{y}; u^0) d\eta. \quad (23)$$

This step is a continuous version of the attention layer in [52]. It looks simple and uses two integral transformations to get the attention score through a softmax operator. This score function has values between $[0, 1]$ and it is multiplied with the $V^0(\eta, \mathbf{y}; u^0)$, which are the extracted features from the input data through another integral transformation.

3.3.2 Solution to Subproblem (17) and (21)

For $u^{2/M}$ and u^1 in problem (17) and (21), they are in the form of

$$u - v = \partial I_{S_1(\sigma_1, \sigma_2)}(u) \quad (24)$$

for the given function $v(\mathbf{x}, \mathbf{y})$ and constants σ_1, σ_2 . It is easy to derive that the solution u of (24) solves the following problem:

$$u = \underset{\bar{u} \in S_1(\sigma_1, \sigma_2)}{\operatorname{argmin}} \frac{1}{2} \|\bar{u} - v\|_{\Omega_y}^2. \quad (25)$$

This shows that u is a projection of v to the set $S_1(\sigma_1, \sigma_2)$. The following theorem gives a closed-form solution formula for u :

Theorem 3.1. *The solution to problem (25) is given as:*

$$u(\mathbf{x}, \mathbf{y}) = \frac{v(\mathbf{x}, \mathbf{y}) - \alpha(\mathbf{x})}{\sqrt{\beta(\mathbf{x})}} \sigma_2 + \sigma_1, \quad (26)$$

$$\text{with } \alpha(\mathbf{x}; v) = \frac{1}{|\Omega_y|} \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi, \quad \beta(\mathbf{x}; v) = \frac{1}{|\Omega_y|} \int_{\Omega_y} (v(\mathbf{x}, \xi) - \alpha(\mathbf{x}))^2 d\xi. \quad (27)$$

Theorem 3.1 is proved in Appendix B. We remark that the expression (27) recovers the normalization layer in the continuous setting. According to Theorem 3.1, we see that the solutions $u^{2/M}$ and $u^{(4+J)/M}$ are explicitly given as:

$$u^{2/M}(\mathbf{x}, \mathbf{y}) = \frac{u^{1/M}(\mathbf{x}, \mathbf{y}) - \alpha(\mathbf{x}; u^{1/M})}{\sqrt{\beta(\mathbf{x}; u^{1/M})}} \sigma_2^0 + \sigma_1^0, \quad (28)$$

$$u^1(\mathbf{x}, \mathbf{y}) = \frac{u^{(3+J)/M}(\mathbf{x}, \mathbf{y}) - \alpha(\mathbf{x}; u^{(3+J)/M})}{\sqrt{\beta(\mathbf{x}; u^{(3+J)/M})}} \sigma_2^0 + \sigma_1^0. \quad (29)$$

3.3.3 Solution to Subproblem (18) and (19)

Problem (18) is a semi-implicit equation for $u^{(2+j)/M}$. It can be solved exactly by the following sequential splitting

$$\begin{cases} \bar{u}^{(2+j)/M} = u^{(1+j)/M} + \left\langle W_j^0(\cdot, \mathbf{y}), u^{(1+j)/M}(\mathbf{x}, \cdot) \right\rangle_{\Omega_y} + b_j^0(\mathbf{x}), \\ u^{(2+j)/M} - \bar{u}^{(2+j)/M} = \partial I_{S_2}(u^{(2+j)/M}). \end{cases} \quad (30)$$

The first equation in (30) is linear in $\bar{u}^{(2+j)/M}$, corresponding to a linear layer in a feed-forward network in continuous settings. The second equation solves

$$u^{(2+j)/M} = \operatorname{argmin}_{v \in S_1} \int_{\Omega_x} \int_{\Omega_y} |v - \bar{u}^{(2+j)/M}|^2 d\mathbf{y} d\mathbf{x}. \quad (31)$$

It is easy to see that its solution can be computed point-wisely:

$$u^{(2+j)/M} = \max\{\bar{u}^{(2+j)/M}, 0\} = \operatorname{ReLU}(\bar{u}^{(2+j)/M}), \quad (32)$$

which corresponds ReLU activation. As a result, Substep $2+j$ for $j = 1, \dots, J-1$ is a feedforward network layer activated by ReLU. Similarly, Substep $2+J$ in (19) is just a linear feedforward layer.

3.4 Spatial Discretizations

In this subsection, we discretize the solution discussed in Section 3.3 spatially. We will show that after spatial discretizations, the splitting scheme (16)–(21) exactly recovers the Transformer architecture in [52].

Suppose $\Omega_x = [0, L_x]$, $\Omega_y = [0, L_y]$. We uniformly discretize Ω_x, Ω_y by N_x and N_y grids in the $\mathbf{x}$ and $\mathbf{y}$ coordinates, respectively. For simplicity, we suppose $L_x = N_x, L_y = N_y$, leading to discretization steps $\Delta x = L_x/N_x = 1$, $\Delta y = L_y/N_y = 1$, and the grids by $\mathbf{x}_k = k, \mathbf{y}_k = k$. We use $\mathbf{1}_m$ to denote an m -dimensional vector with elements 1. For a matrix $\mathbf{W} \in \mathbb{R}^{N_x \times N_y}$ and a vector $\mathbf{b} \in \mathbb{R}^{N_x}$, the matrix $\mathbf{z} = (\mathbf{W} + \mathbf{b}) \in \mathbb{R}^{N_x \times N_y}$ is computed so that

$$\mathbf{z}_{k,l} = W_{k,l} + b_k.$$

This is an easy notation to add a vector to a matrix in the discrete setting. With the spatial discretizations above, we define the discrete integrals (sum) as

$$\int_{\bar{\Omega}_x} f d\mathbf{x} = \sum_{k=1}^{N_x} f_k, \quad \int_{\bar{\Omega}_x} \int_{\bar{\Omega}_y} f(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} = \sum_{k=1}^{N_x} \sum_{l=1}^{N_y} f_{k,l}. \quad (33)$$

For any functions $v(\mathbf{x}), w(\mathbf{x})$ defined on Ω_x , denote their discrete counterpart by $\mathbf{v}, \mathbf{w}$. We denote the discretized inner product between $\mathbf{v}$ and $\mathbf{w}$ as

$$\langle \mathbf{v}, \mathbf{w} \rangle_{\bar{\Omega}_x} = \mathbf{v}^\top \mathbf{w} = \sum_{k=1}^{N_x} v_k w_k. \quad (34)$$

The discretized integrals and inner products $\langle \cdot, \cdot \rangle_{\bar{\Omega}_y}$ and $\langle \cdot, \cdot \rangle_{\bar{\Omega}_x \times \bar{\Omega}_y}$ are defined similarly.

3.4.1 Computing the Discrete Solutions $u^{1/M}$ in (23)

Before we discretize the formula (23), we first discretize some related quantities and operations. To compute the attention scores, we discretize the integral transformations as follows:

$$\mathbf{Q}^0(\mathbf{u}^0) = \mathbf{u}^0 \mathbf{W}^{Q,0}, \quad \mathbf{K}^0(\mathbf{u}^0) = \mathbf{u}^0 \mathbf{W}^{K,0}, \quad \mathbf{V}^0(\mathbf{u}^0) = \mathbf{u}^0 \mathbf{W}^{V,0}. \quad (35)$$

Here, $\mathbf{u}^0$ is in $\mathbb{R}^{N_x \times N_y}$ and $\mathbf{W}^{Q,0}, \mathbf{W}^{K,0}, \mathbf{W}^{V,0}$ are in $\mathbb{R}^{N_y \times N_y}$, and the expressions such as $\mathbf{u}^0 \mathbf{W}^{Q,0}$ represent standard matrix multiplications that yield new matrices in $\mathbb{R}^{N_x \times N_y}$. We emphasize that the multiplication used here is the conventional matrix product. For $\mathbf{A} = \{a_{k,l}\}_{k,l} \in \mathbb{R}^{N_x \times N_x}$, the discretized version of $\text{Softmax}_2(\mathbf{A})$ is given as

$$(\text{Softmax}_{2,\text{dis}}(\mathbf{A}))_{k,l} = \frac{\exp(a_{k,l})}{\sum_{l=1}^{N_x} \exp(a_{k,l})}. \quad (36)$$

Then the updating formula (23) for $u^{1/M}$ in the discrete setting is

$$\mathbf{u}^{1/M} = \mathbf{u}^0 + \text{Softmax}_{2,\text{dis}} \left(\mathbf{Q}^0(\mathbf{u}^0) (\mathbf{K}^0(\mathbf{u}^0))^\top \right) \mathbf{V}^0(\mathbf{u}^0). \quad (37)$$

We emphasize again that this is exactly the attention layer in [52]. This is just a compact mathematical expression for it. All the matrices $\mathbf{W}^{Q,0}, \mathbf{W}^{K,0}, \mathbf{W}^{V,0}$ will be learned in the training process.

3.4.2 Computing the Discrete Solutions $u^{2/M}$ and u^1 in (24)

In Theorem 3.1, for any $\mathbf{v} = \{v_{k,l}\}_{k,l} \in \mathbb{R}^{N_x \times N_y}$, we discretize $\alpha(\mathbf{x})$ and $\beta(\mathbf{x})$ as

$$\alpha_k(\mathbf{v}) = \frac{1}{L_y} \sum_{l=1}^{N_y} v_{k,l}, \quad \beta_k(\mathbf{v}) = \frac{1}{L_y} \sum_l (v_{k,l} - \alpha_k(\mathbf{v}))^2 \quad (38)$$

for $k = 1, \dots, N_x$. Then we compute the discrete solutions $u^{2/M}$ and u^1 as

$$u_{k,l}^{2/M} = \sigma_2^0 \frac{u_{k,l}^{1/M} - \alpha_k(\mathbf{u}^{1/M})}{\sqrt{\beta_k(\mathbf{u}^{1/M})}} + \sigma_1^0, \quad (39)$$

$$u_{k,l}^1 = \sigma_2^0 \frac{u_{k,l}^{(3+J)/M} - \alpha_k(\mathbf{u}^{(3+J)/M})}{\sqrt{\beta_k(\mathbf{u}^{(3+J)/M})}} + \sigma_1^0, \quad (40)$$

for $k = 1, \dots, N_x$ and $l = 1, \dots, N_y$.

3.4.3 Computing the Discrete Solutions $u^{(3+J)/M}$ in (20)

For $u^{(3+J)/M}$, its discrete analogue is computed as

$$\mathbf{u}^{(3+J)/M} = \frac{1}{2} (\mathbf{u}^{(2+J)/M} + \mathbf{u}^{2/M}). \quad (41)$$

3.4.4 Computing the Discrete Solutions $u^{(2+j)/M}$ in (30)

In (30), we first compute

$$\bar{\mathbf{u}}^{(2+j)/M} = \mathbf{u}^{(1+j)/M} + \mathbf{u}^{(1+j)/M} \mathbf{W}_j^0 + \mathbf{b}_j^0, \quad (42)$$

and then update

$$\mathbf{u}^{(2+j)/M} = \text{ReLU}(\bar{\mathbf{u}}^{(2+j)/M}). \quad (43)$$

The structure of the discretized scheme (16)–(21) is illustrated in Figure 1(b), and it exactly recovers the well-known Transformer architecture introduced in [52], see the next section for detailed explanations.

4 Mathematical Explanations of Transformers

4.1 Connections to Transformer Encoder in [52]

Transformer architecture was first proposed in [52] for natural language processing. After time and spatial discretization, our splitting scheme recovers the well-known Transformer with single-head attention in [52]. We supply the details of the explanations in the following. Let us first show that one iteration $\mathbf{u}^0 \rightarrow \mathbf{u}^1$ realizes one Transformer block:

- Input $\mathbf{u}^0$: With our discretization, $\mathbf{u}^0 \in \mathbb{R}^{N_x \times N_y}$. It is an input with N_x tokens. Each token is represented as a vector with N_y numbers.
- Scaled dot-product attention: Equation (37) is a single head attention with a skip connection. The second term in the right-hand side of (37) is a single-head attention with query matrix $\mathbf{Q}^0$, key matrix $\mathbf{K}^0$ and value matrix $\mathbf{V}^0$, whose weight matrices are $\mathbf{W}^{Q,0}$, $\mathbf{W}^{K,0}$ and $\mathbf{W}^{V,0}$, respectively. It recovers the single-head attention in [52]. The first term in the right-hand side is a skip connection, adding together the input and output of attention.
- Layer normalization: Equation (39)–(40) perform layer normalization. For each token, (39)–(40) normalize the input along the embedding direction to have mean σ_1 and variance σ_2^2 .
- Position-wise feedforward network: Equation (42)–(43) realizes a feedforward network with J layers activated by ReLU. In particular, (42) can be rewritten as

$$\bar{\mathbf{u}}^{(2+j)/M} = \mathbf{u}^{(1+j)/M} (\mathbf{I} + \mathbf{W}_j^0) + \mathbf{b}_j^0,$$

where $\mathbf{I}$ denotes the identity matrix. It is a linear layer with weight matrix $(\mathbf{I} + \mathbf{W}_j^0)$ and bias $\mathbf{b}_j^0$. Set $J = 2$. Equation (42)–(43) recovers the feedforward network in [52].

- Skip connection: Equation (41) is a relaxation step in the splitting scheme. It realizes the skip connection by averaging the input and output of the feedforward network.

The discussion above shows that one time step of the splitting scheme supplied in Section 3.2 realizes the Transformer of [52]. In Section 3.2, the time domain is discretized into N_t time steps. Thus, the whole operator-splitting scheme is equivalent to compositions of N_t Transformer blocks. Since the learnable variables $\theta^n = \{\mathbf{W}^{Q,n}, \mathbf{W}^{K,n}, \mathbf{W}^{V,n}, \{\mathbf{W}_j^n, \mathbf{b}_j^n\}_{j=1}^J\}$ depends on time, they are different for different time steps. Setting $N_t = 6$ recovers the architecture in [52]. Furthermore, solving (15) for the control variables is equivalent to training the network.

4.2 Connections to ViT in [10]

Based on the Transformer encoder in [52], vision Transformer (ViT) was proposed in [10] for image classification. In ViT, each image is cropped into patches, each of which is taken as a token. ViT has a similar architecture as the Transformer encoder in [52], except that it has an additional input embedding layer with learnable parameters, and a linear layer at the end for output.

In order to use (16)–(21) to provide a mathematical explanation of ViT, we need to incorporate the additional embedding layer and last linear layer with our algorithm. It can be easily done by introducing data-driven data pre-processing and post-processing steps. For the additional input embedding, we design a pre-processing step to generate the initial condition f . Consider the discrete setting. Denote $\mathbf{R} \in \mathbb{R}^{(N_x-1) \times D}$ as the collection of $(N_x - 1)$ patches of an image, where each row of $\mathbf{R}$ corresponds to one flattened patch of dimension D . Let $\mathbf{E} \in \mathbb{R}^{D \times N_y}$ be an embedding matrix and $\mathbf{v}_0 \in \mathbb{R}^{1 \times N_y}$ be a vector where both are learnable. The vector $\mathbf{v}_0$ represents the extra learnable embedding. We generate $\mathbf{u}_0$ as

$$\mathbf{u}_0 = F(\mathbf{R}) = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{R}\mathbf{E} \end{bmatrix}. \quad (44)$$

The last linear layer in ViT can be taken as a data post-processing. For any $\mathbf{u} \in \mathbb{R}^{N_x \times N_y}$ and weight matrix $\mathbf{B} \in \mathbb{R}^{N_y \times d}$, define

$$G(\mathbf{B}; \mathbf{u}) = \begin{bmatrix} 1 & \mathbf{0}_{1 \times (N_x-1)} \end{bmatrix} \mathbf{u} \mathbf{B}, \quad (45)$$

where $\mathbf{0}_{1 \times (N_x-1)}$ is a zero vector of size $(N_x - 1)$. Let $\bar{\theta}, \bar{\mathcal{N}}$ be defined as in Section 3.1. We denote

$$\tilde{\theta} = \{\bar{\theta}, \mathbf{v}_0, \mathbf{E}, \mathbf{B}\}$$

as the collection of all learnable parameters. With the data pre- and post-processing, we define the whole operation

$$\bar{\mathcal{N}}_{\tilde{\theta}}^{\text{ViT}} = G \circ \bar{\mathcal{N}} \circ F. \quad (46)$$

One can readily check that $\bar{\mathcal{N}}_{\tilde{\theta}}^{\text{ViT}}$ recovers the ViT in [10]. Given a dataset $\{(\mathbf{u}_i, \mathbf{v}_i)\}_{i=1}^B$, we learn all parameters by solving

$$\min_{\tilde{\theta}} \frac{1}{B} \sum_{i=1}^B \ell(\bar{\mathcal{N}}_{\tilde{\theta}}^{\text{ViT}}(\mathbf{u}_i), \mathbf{v}_i), \quad (47)$$

which is equivalent to training a ViT.

5 Multi-Head Attention

As discussed in Section 4, the proposed splitting scheme for problem (10) is the Transformer architecture with one-head attention. In this section, we extend (10) and splitting scheme (16)–(21) so that it realizes multi-head attention.

5.1 Control Problem with Continuous Head Dimension

We first consider a continuous-head setting. After discretizations, we will show that it recovers multi-head attention. In the continuous-head setting, we take head as an additional dimension. To accommodate the new dimension, we update the definition of W^Q, W^K, W^V to

$W^Q(\mathbf{y}, \tilde{\mathbf{y}}, h, t)$, $W^K(\mathbf{y}, \tilde{\mathbf{y}}, h, t)$, $W^V(\mathbf{y}, \tilde{\mathbf{y}}, h, t)$ on $\Omega_y \times \Omega_y \times \Omega_h \times [0, +\infty)$, where h is the variable for the head dimension with domain Ω_h . Similar to (3)–(5), we define

$$\begin{aligned} Q(\mathbf{x}, \mathbf{y}, h, t; u) &= \left\langle W^Q(\cdot, \mathbf{y}, h, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y}, \\ K(\mathbf{x}, \mathbf{y}, h, t; u) &= \left\langle W^K(\cdot, \mathbf{y}, h, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y}, \\ V(\mathbf{x}, \mathbf{y}, h, t; u) &= \left\langle W^V(\cdot, \mathbf{y}, h, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y}. \end{aligned} \quad (48)$$

We update $\gamma(\mathbf{x}, \tilde{\mathbf{x}}, t; u)$ in (6) to

$$\gamma(\mathbf{x}, \tilde{\mathbf{x}}, h, t; u) = \text{Softmax}_2 \left(\frac{1}{\sqrt{|\Omega_y|}} \left\langle Q(\mathbf{x}, \cdot, h, t; u), K(\tilde{\mathbf{x}}, \cdot, h, t; u) \right\rangle_{\Omega_y} \right). \quad (49)$$

The control problem (10) is updated to

$$\begin{cases} u_t = \underbrace{\int_{\Omega_h} \left\langle \gamma(\mathbf{x}, \cdot, h, t; u), V(\cdot, \mathbf{y}, h, t; u) \right\rangle_{\Omega_x} dh}_{\text{I-a: attention}} + \underbrace{\frac{\partial I_{S_1(\sigma_1, \sigma_2)}(u)}{\partial u}}_{\text{II: layer normalization}} \\ \quad + \underbrace{\sum_{j=1}^J \left(\left\langle W_j(\cdot, \mathbf{y}, t), u(\mathbf{x}, \cdot, t) \right\rangle_{\Omega_y} + b_j(\mathbf{x}, t) \right)}_{\text{III: fully connected network}} + \partial I_{S_2}(u) \text{ for } t \in (0, T], \\ u(\mathbf{x}, \mathbf{y}, 0) = f(\mathbf{x}, \mathbf{y}). \end{cases} \quad (50)$$

Compared to (10), problem (50) replaces I by I-a, which introduces an additional integral with respect to h .

5.2 Time Discretizations Through Operator-Splitting Method

The time discretizations and operator-splitting scheme for problem (50) are the same as those for problem (10) except that we update Substep 1 to

$$u^{1/M} - u^0 = \int_{\Omega_h} \left\langle \gamma^0(\mathbf{x}, \cdot, h; u^0), V^0(\cdot, \mathbf{y}, h; u^0) \right\rangle_{\Omega_x} dh, \quad (51)$$

whose solution is given as

$$u^{1/M} = u^0 + \int_{\Omega_h} \int_{\Omega_x} \text{Softmax}_2 \left(\frac{1}{\sqrt{|\Omega_y|}} \left\langle Q^0(\mathbf{x}, \cdot, h; u^0), K^0(\eta, \cdot, h; u^0) \right\rangle_{\Omega_y} \right) V^0(\eta, \mathbf{y}, h; u^0) d\eta dh. \quad (52)$$

5.3 Discrete Solutions of $u^{1/M}$ in (52)

Suppose $\Omega_h = [0, L_h]$. In addition to the spatial discretization discussed in Section 3.4, we discretize the head domain into N_h grids. For simplicity, we suppose $L_h = N_h$.

For function $W^{K,0}(\mathbf{y}, \tilde{\mathbf{y}}, h)$, we denote $W_{k,l,m}^{Q,0} = W^{Q,0}(k, l, m)$, $\mathbf{W}_m^{Q,0} = \{W_{k,l,m}^{Q,0}\}_{k,l} \in \mathbb{R}^{N_y \times N_y}$ and $\mathbf{W}^{Q,0} = \{\mathbf{W}_m^{Q,0}\}_m \in \mathbb{R}^{N_y \times N_y \times N_h}$. We define $\mathbf{W}^{K,0}, \mathbf{W}^{V,0}$ similarly. We denote the discretized integral transformation in the updated attention score as

$$\mathbf{Q}_m^0(\mathbf{u}^0) = \mathbf{u}^0 \mathbf{W}_m^{Q,0}, \quad \mathbf{K}_m^0(\mathbf{u}^0) = \mathbf{u}^0 \mathbf{W}_m^{K,0}, \quad \mathbf{V}_m^n(\mathbf{u}^0) = \mathbf{u}^0 \mathbf{W}_m^{V,0},$$

and denote $\mathbf{Q}^0(\mathbf{u}) = \{\mathbf{Q}_m^0(\mathbf{u}^0)\}_m$, $\mathbf{K}^0(\mathbf{u}^0) = \{\mathbf{K}_m^0(\mathbf{u}^0)\}_m$, $\mathbf{V}^0(\mathbf{u}^0) = \{\mathbf{V}_m^0(\mathbf{u}^0)\}_m \in \mathbb{R}^{N_x \times N_y \times N_h}$. The inner product $\left\langle Q^0(\mathbf{x}, \cdot, h; u^0), K^0(\tilde{\mathbf{x}}, \cdot, h; u^0) \right\rangle_{\Omega_y}$ is discretized as

$$\left\langle \mathbf{Q}_m^0(\mathbf{u}^0), \mathbf{K}_m^0(\mathbf{u}^0) \right\rangle_{\Omega_y} = \mathbf{Q}_m^0(\mathbf{u}^0)(\mathbf{K}_m^0(\mathbf{u}^0))^\top. \quad (53)$$

Then the updating formula (52) for $u^{1/M}$ is discretized as

$$\mathbf{u}^{1/M} = \mathbf{u}^0 + \sum_{m=1}^{N_h} \text{Softmax}_{2,\text{dis}} \left(\frac{1}{\sqrt{L_x}} \mathbf{Q}_m^0(\mathbf{u}^0)(\mathbf{K}_m^0(\mathbf{u}^0))^\top \right) \mathbf{V}_m^0(\mathbf{u}^0). \quad (54)$$

5.4 Connections to Transformers

Similar to the discussion in Section 4, with (54), the operator-splitting scheme for the control problem (50) realizes N_t times encoder in [52] with multi-head attention. To show this, we only need to demonstrate the equivalence between (54) and the multi-head attention.

The second term in the right-hand side of (54) sums over the embedding dimension and head dimension. It is a multi-head attention with N_h heads, query matrix $\mathbf{Q}^0$, key matrix $\mathbf{K}^0$ and value matrix $\mathbf{V}^0$, whose weight matrices are $\mathbf{W}^{Q,0}$, $\mathbf{W}^{K,0}$ and $\mathbf{W}^{V,0}$, respectively.

6 Convolution Transformers for Video and Image Problems

Image and video data inherently possess rich local spatial structures, which have been effectively exploited by convolutional neural networks (CNNs) [43]. Convolutions serve as a powerful mechanism for capturing spatially localized patterns, making them especially well-suited for data defined on Cartesian grids.

In contrast, Transformer architectures were originally developed for language processing, where the data—represented by functions such as $u(\mathbf{x}, \mathbf{y}, t)$ —often lack explicit spatial structure, as in the case of sentences or token sequences. For such unstructured data, attention-based mechanisms leveraging general integral transforms offer a flexible and effective modeling tool.

However, when applying Transformers to structured data such as images, videos, or other signals on regular grids, it has been shown that convolutional operations are significantly more efficient for feature extraction. This is due to their locality, weight sharing, and computational efficiency. Notably, convolutions can be interpreted as a special class of integral transformations with translation-invariant kernels and localized support.

Thanks to the integral formulation of the Q , K , and V operators in our framework (Sections 2–3), it becomes straightforward to specialize these operators to convolutions. This allows our model to seamlessly incorporate convolutional structures, thereby leveraging domain-specific inductive biases and enhancing computational performance when applied to grid-structured data.

6.1 Convolution Transformers

We discuss the case for grayscale images. The framework can be extended to color images and videos by introducing additional variables to u .

We consider functions of interest $u(x, \mathbf{y}, t)$ where $x \in \Omega_x \subset \mathbb{R}$ is the index of tokens (image patches), $\mathbf{y} \in \Omega_y \subset \mathbb{R}^2$ are variables for spatial domain (indices of pixel location), and t is the time. For each token and any fixed time t , u is a two-dimensional image.

In this setting, we define $W^Q(\mathbf{y}, t)$, $W^K(\mathbf{y}, t)$, $W^V(\mathbf{y}, t) \in \Omega_y \times [0, +\infty)$ as convolution kernels.

Operations Q, K, V are defined by covolution as

$$\begin{aligned} Q(x, \mathbf{y}, t; u) &= W^Q(\cdot, t) * u(x, \cdot, t) = \int_{\Omega_y} W^Q(\xi, t) u(x, \mathbf{y} - \xi, t) d\xi, \\ K(x, \mathbf{y}, t; u) &= W^K(\cdot, t) * u(x, \cdot, t) = \int_{\Omega_y} W^K(\xi, t) u(x, \mathbf{y} - \xi, t) d\xi, \\ V(x, \mathbf{y}, t; u) &= W^V(\cdot, t) * u(x, \cdot, t) = \int_{\Omega_y} W^V(\xi, t) u(x, \mathbf{y} - \xi, t) d\xi. \end{aligned} \quad (55)$$

The score function is computed as

$$\gamma(x, \tilde{x}, t; u) = \text{Softmax}_2 \left(\frac{1}{\sqrt{|\Omega_y|}} \int_{\Omega_y} Q(x, \xi, t; u) K(\tilde{x}, \xi, t; u) d\xi \right), \quad (56)$$

and the output of the attention layer is then given by: $\langle \gamma(x, \cdot, t; u), V(\cdot, \mathbf{y}, t; u) \rangle_{\Omega_x}$.

With the modification above, scheme (16)–(18) gives rise to convolutional Transformer. Solvers discussed in Section 3.3 can still be used to solve subproblems in the new scheme.

6.2 Connections to CvT in [56]

Convolutional vision Transformer (CvT) [56] incorporates Vit with convolutional neural networks (CNN), which utilizes both image local spatial structures (by CNN) and global contexture (by Transformer). CvT consists of two components, convolutional token embedding and convolutional Transformer block. The convolution Transformer discussed in Section 6.1 realizes a one-head CvT with one stage and without convolutional token embedding.

In fact, the convolutional token embedding is an additional convolution layer. Our operator-splitting scheme can be easily revised to accomodate this layer. Let $W^C(\mathbf{x}, \mathbf{y}, t)$ be a convolution kernel and $b^C(\mathbf{x}, t)$ be a bias term. We consider the following control problem

$$\begin{cases} u_t = \underbrace{\langle \gamma(\mathbf{x}, \cdot, t; u), V(\cdot, \mathbf{y}, t; u) \rangle_{\Omega_x}}_{\text{I: attention}} + \underbrace{\partial I_{S_1(\sigma_1(t), \sigma_2(t))}(u)}_{\text{II: layer norml.}} + \underbrace{W^C(\cdot, t) * u(\mathbf{x}, \cdot, t) + b^C(\mathbf{x}, t)}_{\text{IV: conv. token embedding}} \\ \quad + \underbrace{\sum_{j=1}^J \left(\langle W_j(\cdot, \mathbf{y}, t), u(\mathbf{x}, \cdot, t) \rangle_{\Omega_y} + b_j(\mathbf{x}, t) \right)}_{\text{III: fully connected network}} + \partial I_{S_2}(u), \quad t \in (0, T], \\ u(\mathbf{x}, \mathbf{y}, 0) = f(\mathbf{x}, \mathbf{y}), \end{cases} \quad (57)$$

where the score function γ is computed as discussed in Section 6.1. For the operator-splitting scheme (16)–(18), we add two additional substeps before (16). Given u^0 , we compute u^1 by $M = 6 + J$ substeps:

Substep 1: Solve $\tilde{u}^{1/M}$ from

$$u^{1/M} - u^0 = W^{C,0}(\cdot) * u^0(\mathbf{x}, \cdot) + b^{C,0}(\mathbf{x}). \quad (58)$$

Substep 2: Compute $u^{2/M}$ from

$$u^{2/M} - u^{1/M} = \partial I_{S_1(\sigma_1^0, \sigma_2^0)}(u^{1/M}). \quad (59)$$

The remaining intermediate variables $u^{3/M}, \dots, u^1$ can be computed via (16)–(18), respectively. In spatial discretization, N_x corresponds to the number of tokens, i.e., the number of convolution

channels in CvT. Equipped with data pre- and post-processing as discussed in Section 4.2, the discretized new operator-splitting scheme recovers a one-head one-stage CvT.

For a K -stage CvT, the number of tokens changes from stage to stage. This property can be realized by incorporating the new operator-splitting scheme with a hybrid splitting scheme [48]. Specifically, we need to define K convolution kernels and a bias for convolutional token embedding. Based on the multigrid idea, these kernels are discretized at different scales, corresponding to various numbers of tokens.

7 Conclusion

In this paper, we have introduced a novel operator-theoretic framework that interprets the Transformer architecture as a discretized operator-splitting scheme for a continuous integro-differential equation. Within this formulation, core components of the Transformer—such as self-attention, layer normalization, and feedforward networks—are rigorously derived as substeps in a structured variational process and operator-splitting techniques. This approach establishes a unified mathematical foundation that applies broadly to Transformer-based models, including the original Transformer [52], Vision Transformer (ViT) [10], and Convolutional vision Transformer (CvT) [56]. By bridging deep learning architectures with continuous mathematical modeling, our framework advances theoretical understanding and offers a principled pathway for the design, analysis, and control of future neural architectures.

Our continuous formulation offers a principled basis for deriving new architectures through alternative numerical discretization schemes, potentially leading to models with improved stability. It also enables the use of PDE theory to analyze Transformer properties such as stability, approximation power, and convergence. Furthermore, framing the network as an optimal control problem links deep learning to control theory, which may inspire new optimization algorithms and insights into training dynamics.

Our framework currently considers ReLU activation in the feedforward layer and only focuses on the transformer block of [52]. Following this work, possible future directions include generalizing this framework to incorporate general activation functions, incorporation of discrete inductive biases such as positional encodings into the continuous formulation, and a more rigorous analysis of the well-posedness and regularity of the underlying integro-differential equation.

We believe the continuous operator perspective introduced here not only deepens the theoretical understanding of Transformers but also provides a flexible and principled foundation for the next generation of attention-based models. By continuing to explore the intersection of continuous dynamical systems, operator splitting, and deep learning, we hope to further bridge the gap between applied architecture design and rigorous mathematical analysis.

Appendix

A Explanation of the variables for Transformers

The input to the Transformer model is usually a sequence. In the context of language processing, the input is usually sentences. Transformers do not process raw text directly. Instead, they break sentences into tokens. A token can be a word (e.g., "hello"), a subword (e.g., "multi" + "grid" = "multigrid"), or even a single character. Then, each different token is transformed into unique embedding vectors through an embedding layer. The way text is tokenized affects how efficiently a model understands language.

For example, we have an input sentence "Life is like a boat", and we treat each word as a token. The embedding layer would transform each token into a 512-dimensional vector. Then,

the input sentence is represented by a matrix U of size 5×512 :

$$\underbrace{\text{Life is like a boat}}_{\text{Input sentence}} \rightarrow \underbrace{["\text{Life}", "\text{is}", "\text{like}", "\text{a}", "\text{boat}"]}_{\text{Tokens}} \rightarrow U = \underbrace{\begin{bmatrix} \text{Embedding}(\text{"Life"}) \\ \text{Embedding}(\text{"is"}) \\ \text{Embedding}(\text{"like"}) \\ \text{Embedding}(\text{"a"}) \\ \text{Embedding}(\text{"boat"}) \end{bmatrix}}_{\text{Embedded matrix}}.$$

In general, a sentence of N_x tokens with N_y -dimensional embedding would be transformed into a matrix U of size $N_x \times N_y$. This matrix U can be viewed as the discretization of a continuous function $u(\mathbf{x}, \mathbf{y}, t) : \Omega_x \times \Omega_y \rightarrow \mathbb{R} \times [0, T)$, where Ω_x and Ω_y are continuous domains, on a structured grid of size $N_x \times N_y$. Here, the variable $\mathbf{x}$ indicates the position of tokens, $\mathbf{y}$ indicates the entry of token vectors, and t indicates the time variable. The embedded matrix U is fed into the Transformer model and updated by attention layers.

B Proof of Theorem 3.1

Proof of Theorem 3.1. Problem (25) can be rewritten as

$$\begin{cases} u(\mathbf{x}, \mathbf{y}) = \operatorname{argmin}_{\bar{u}} \frac{1}{2} \|\bar{u} - v\|_{\Omega_y}^2, \\ \frac{1}{|\Omega_y|} \int_{\Omega_y} u(\mathbf{x}, \xi) d\xi = \sigma_1, \\ \frac{1}{|\Omega_y|} \int_{\Omega_y} (u(\mathbf{x}, \xi) - \sigma_1)^2 d\xi = \sigma_2^2. \end{cases} \quad (60)$$

Introducing Lagrange multipliers λ_1, λ_2 , the minimizers for the above problem satisfy:

$$\begin{cases} (u - v) + \frac{\lambda_1}{|\Omega_y|} + \frac{2\lambda_2}{|\Omega_y|} (u - \sigma_1) = 0, \\ \frac{1}{|\Omega_y|} \int_{\Omega_y} u(\mathbf{x}, \mathbf{y}) d\mathbf{y} = \sigma_1, \\ \frac{1}{|\Omega_y|} \int_{\Omega_y} (u(\mathbf{x}, \mathbf{y}) - \sigma_1)^2 d\mathbf{y} = \sigma_2^2. \end{cases} \quad (61)$$

Integrate the first equation over $\mathbf{y}$ gives rise to

$$0 = \int_{\Omega_y} u(\mathbf{x}, \xi) d\xi - \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi + \lambda_1 = |\Omega_y| \sigma_1 - \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi + \lambda_1, \quad (62)$$

implying that

$$\lambda_1 = \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi - |\Omega_y| \sigma_1. \quad (63)$$

Multiplying the first equation in (61) by $(u - \sigma_1)$ and then integrating over $\mathbf{y}$ give rise to

$$\int_{\Omega_y} (u(\mathbf{x}, \xi) - v(\mathbf{x}, \xi))(u(\mathbf{x}, \xi) - \sigma_1) d\xi + 2\lambda_2 \sigma_2^2 = 0, \quad (64)$$

from which we deduce

$$\int_{\Omega_y} (u^2(\mathbf{x}, \xi) - u(\mathbf{x}, \xi) \sigma_1) d\xi - \int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi + 2\lambda_2 \sigma_2^2 = 0. \quad (65)$$

Note that

$$\begin{aligned}
\int_{\Omega_y} (u^2(\mathbf{x}, \xi) - u(\mathbf{x}, \xi)\sigma_1) d\xi &= \int_{\Omega_y} (u^2(\mathbf{x}, \xi) - 2u(\mathbf{x}, \xi)\sigma_1 + \sigma_1^2) d\xi \\
&= \int_{\Omega_y} (u(\mathbf{x}, \xi) - \sigma_1)^2 d\xi = |\Omega_y| \sigma_2^2. \quad (66)
\end{aligned}$$

Substituting (66) into (65) gives

$$\lambda_2 = -\frac{|\Omega_y|}{2} + \frac{\int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi}{2\sigma_2^2}. \quad (67)$$

Substituting (63) and (67) into the first equation of (61), we get

$$\begin{aligned}
0 &= (u - v) + \frac{\lambda_1}{|\Omega_y|} + \frac{2\lambda_2}{|\Omega_y|}(u - \sigma_1) \\
&= (u - v) + \frac{1}{|\Omega_y|} \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi - \sigma_1 + \left(-1 + \frac{\int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi}{|\Omega_y| \sigma_2^2} \right) u \\
&\quad - \left(-1 + \frac{\int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi}{|\Omega_y| \sigma_2^2} \right) \sigma_1 \\
&= \frac{\int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi}{|\Omega_y| \sigma_2^2} (u - \sigma_1) - v + \frac{1}{|\Omega_y|} \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi. \quad (68)
\end{aligned}$$

We have

$$\frac{\int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi}{|\Omega_y| \sigma_2^2} (u - \sigma_1) = v - \frac{1}{|\Omega_y|} \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi. \quad (69)$$

Define $\alpha(\mathbf{x}), \beta(\mathbf{x})$ as in Theorem 3.1. We claim that the solution to (69) is given as

$$u = \frac{v - \alpha}{\sqrt{\beta}} \sigma_2 + \sigma_1. \quad (70)$$

To verify it, note that (70) can be rewritten as

$$\frac{u - \sigma_1}{\sigma_2} = \frac{v - \alpha}{\sqrt{\beta}}. \quad (71)$$

Utilizing (71), we deduce

$$\begin{aligned}
&\frac{\int_{\Omega_y} v(\mathbf{x}, \xi)(u(\mathbf{x}, \xi) - \sigma_1) d\xi}{|\Omega_y| \sigma_2^2} (u - \sigma_1) \\
&= \frac{1}{|\Omega_y|} \left(\int_{\Omega_y} v(\mathbf{x}, \xi) \frac{(u(\mathbf{x}, \xi) - \sigma_1)}{\sigma_2} d\xi \right) \frac{u - \sigma_1}{\sigma_2} \\
&= \frac{1}{|\Omega_y|} \left(\int_{\Omega_y} v(\mathbf{x}, \xi) \frac{(v(\mathbf{x}, \xi) - \alpha)}{\sqrt{\beta}} d\xi \right) \frac{v - \alpha}{\sqrt{\beta}} \\
&= \frac{1}{|\Omega_y| \beta} \left(\int_{\Omega_y} [v^2(\mathbf{x}, \xi) - 2\alpha v(\mathbf{x}, \xi) + \alpha^2] d\xi \right) (v - \alpha) \\
&= \frac{1}{|\Omega_y| \beta} \left(\int_{\Omega_y} (v(\mathbf{x}, \xi) - \alpha)^2 d\xi \right) (v - \alpha) \\
&= \frac{1}{|\Omega_y| \beta} (|\Omega_y| \beta) (v - \alpha) \\
&= v - \frac{1}{|\Omega_y|} \int_{\Omega_y} v(\mathbf{x}, \xi) d\xi, \quad (72)
\end{aligned}$$

where the second equality follows from (71). The derivation above verifies that (70) is the solution to (69). $\square$

C An Introduction to Operator Splitting Schemes

Operator-splitting methods decompose a complicated time-evolution problem into several substeps so that each substep can be solved efficiently. Consider the evolution equation

$$\begin{cases} u_t + \sum_{k=1}^K A_k(t; u) = 0 \text{ in } \Omega \times (0, T], \\ u(0) = u_0, \end{cases} \quad (73)$$

where $A_k(t; u)$'s are operators applied on u . Denote $t^n = n\Delta t$. Given u^n , there are three types of splitting strategies to compute u^{n+1} .

Sequential Lie scheme [16, Sec. 2.2] decomposes this computation into K sequential substeps. Given u^n , one computes u^{n+1} as follows:

For $k = 1, \dots, K$, solve

$$\begin{cases} u_t + A_k(t; u) = 0 \text{ in } \Omega \times (t^n, t^{n+1}], \\ u(t^n) = u^{n+(k-1)/K}, \end{cases} \quad (74)$$

and set $u^{n+k/K} = u(t^{n+1})$.

Parallel splitting [37] decomposes the computation into K parallel substeps:

For $k = 1, \dots, K$, compute

$$\begin{cases} v_t + K A_k(t; v) = 0 \text{ in } \Omega \times (t^n, t^{n+1}], \\ v(t^n) = u^n, \end{cases} \quad (75)$$

and set $v_k = u(t^{n+1})$. Then compute

$$u^{n+1} = \frac{1}{K} \sum_{k=1}^K v_k.$$

Hybrid splitting [48] is a combination of sequential and parallel splitting, which decomposes the computation into sequential substeps, each of which contains several parallel substeps.

Substep (74) and (75) can be solved by one-step explicit or implicit scheme. For example, (74) can be solved by

$$\frac{u^{n+k/K} - u^{n+(k-1)/K}}{\Delta t} + A_k(u^{n+k/K}) = 0 \quad \text{or} \quad \frac{u^{n+k/K} - u^{n+(k-1)/K}}{\Delta t} + A_k(u^{n+(k-1)/K}) = 0.$$

Our scheme in Section 3.2 is a sequential splitting scheme, in which (16) uses explicit scheme, and (17) and (21) use implicit scheme. Substep (18) is semi-implicit, which is slightly different from others. In fact, (18) is a composition of two substeps, as it is solved by another sequential splitting (30). In (30), the first substep is explicit and the second one is implicit.

It can be shown that with proper conditions, all three strategies are first-order accurate schemes for the evolution equation.

References

- [1] F. Altekrüger, P. Hagemann, and G. Steidl. Conditional generative models are provably robust: Pointwise guarantees for Bayesian inverse problems. *Transactions on Machine Learning Research*, 2023.

- [2] F. Altekrüger, J. Hertrich, and G. Steidl. Neural wasserstein gradient flows for discrepancies with Riesz kernels. In *International Conference on Machine Learning*, pages 664–690. PMLR, 2023.
- [3] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [4] M. Benning, E. Celledoni, M. J. Ehrhardt, B. Owren, and C.-B. Schönlieb. Deep learning as optimal control problems: Models and numerical methods. *Journal of Computational Dynamics*, 6(2):171–198, 2019.
- [5] A. Bryutkin, J. Huang, Z. Deng, G. Yang, C.-B. Schönlieb, and A. Aviles-Rivero. HAMLET: graph transformer neural operator for partial differential equations. In *Proceedings of the 41st International Conference on Machine Learning*, pages 4624–4641, 2024.
- [6] M. Chen, H. Jiang, W. Liao, and T. Zhao. Nonparametric regression on low-dimensional manifolds using deep ReLU networks: Function approximation and statistical recovery. *Information and Inference: A Journal of the IMA*, 11(4):1203–1253, 2022.
- [7] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
- [8] C.-W. Cheng, J. Huang, Y. Zhang, G. Yang, C.-B. Schönlieb, and A. I. Aviles-Rivero. Mamba neural operator: Who wins? transformers vs. state-space models for PDEs. *Journal of Computational Physics*, page 114567, 2025.
- [9] Y. Cui, Y. Xu, R. Peng, and D. Wu. Layer normalization for TSK fuzzy system optimization in regression problems. *IEEE Transactions on Fuzzy Systems*, 31(1):254–264, 2022.
- [10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [11] S. Dutta, T. Gautam, S. Chakrabarti, and T. Chakraborty. Redesigning the transformer architecture with insights from multi-particle dynamical systems. In *Advances in Neural Information Processing Systems*, volume 34, pages 5531–5544, 2021.
- [12] T. Furuya, M. V. de Hoop, and G. Peyré. Transformers are universal in-context learners. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [13] B. Geshkovski, C. Letrouit, Y. Polyanskiy, and P. Rigollet. A mathematical perspective on transformers. *Bulletin of the American Mathematical Society*, 62(3):427–479, 2025.
- [14] R. Glowinski and P. Le Tallec. *Augmented Lagrangian and Operator-Splitting Methods in Nonlinear Mechanics*, volume 9. Society for Industrial Mathematics, 1989.
- [15] R. Glowinski, S. J. Osher, and W. Yin. *Splitting methods in Communication, Imaging, Science, and Engineering*. Springer, 2017.
- [16] R. Glowinski, T.-W. Pan, and X.-C. Tai. Some facts about operator-splitting and alternating direction methods. In *Splitting Methods in Communication, Imaging, Science, and Engineering*, pages 19–94. Springer, 2016.
- [17] A. Graves, A.-r. Mohamed, and G. Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 6645–6649. IEEE, 2013.

- [18] K. Gregor and Y. LeCun. Learning fast approximations of sparse coding. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, pages 399–406, 2010.
- [19] E. Haber, L. Ruthotto, E. Holtham, and S.-H. Jun. Learning across scales—multiscale methods for convolution neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [20] P. Hagemann, R. Gruhlke, B. Stankewitz, C. Schillings, and G. Steidl. Provable mixed-noise learning with flow-matching. *arXiv preprint arXiv:2508.18122*, 2025.
- [21] P. Hagemann, J. Hertrich, and G. Steidl. Stochastic normalizing flows for inverse problems: A Markov chains viewpoint. *SIAM/ASA Journal on Uncertainty Quantification*, 10(3):1162–1190, 2022.
- [22] A. Havrilla and W. Liao. Understanding scaling laws with statistical and approximation theory for transformer neural networks on intrinsically low-dimensional data. In *Advances in Neural Information Processing Systems*, volume 37, pages 42162–42210, 2024.
- [23] J. He and J. Xu. MgNet: A unified framework of multigrid and convolutional neural network. *Science China Mathematics*, 62:1331–1354, 2019.
- [24] S. Jelassi, M. Sander, and Y. Li. Vision transformers provably learn spatial structure. In *Advances in Neural Information Processing Systems*, volume 35, pages 37822–37836, 2022.
- [25] F. Jiang, Y. Jiang, H. Zhi, Y. Dong, H. Li, S. Ma, Y. Wang, Q. Dong, H. Shen, and Y. Wang. Artificial intelligence in healthcare: past, present and future. *Stroke and Vascular Neurology*, 2(4), 2017.
- [26] K. Kan, X. Li, B. J. Zhang, T. Sahai, S. Osher, K. Kumar, and M. A. Katsoulakis. Stability of transformers under layer normalization. *arXiv preprint arXiv:2510.09904*, 2025.
- [27] D. I. Ketcheson. Relaxation Runge–Kutta methods: Conservation and stability for inner-product norms. *SIAM Journal on Numerical Analysis*, 57(6):2850–2870, 2019.
- [28] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 2012.
- [29] Z. Lai, L.-H. Lim, and Y. Liu. Attention is a smoothed cubic spline. *arXiv preprint arXiv:2408.09624*, 2024.
- [30] Y. Lan, Z. Li, J. Sun, and Y. Xiang. DOSnet as a non-black-box PDE solver: When deep learning meets operator splitting. *Journal of Computational Physics*, 491:112343, 2023.
- [31] Z. Li, N. B. Kovachki, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, et al. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [32] H. Liu, J. Liu, R. H. Chan, and X.-C. Tai. Double-well net for image segmentation. *Multi-scale Modeling & Simulation*, 22(4):1449–1477, 2024.
- [33] Y. Liu, Z. Zhang, and H. Schaeffer. PROSE: Predicting multiple operators and symbolic expressions using multimodal transformers. *Neural Networks*, 180:106707, 2024.
- [34] Z. Long, Y. Lu, X. Ma, and B. Dong. PDE-Net: Learning pdes from data. In *International Conference on Machine Learning*, pages 3208–3216. PMLR, 2018.

- [35] J. Lu, Z. Shen, H. Yang, and S. Zhang. Deep network approximation for smooth functions. *SIAM Journal on Mathematical Analysis*, 53(5):5465–5506, 2021.
- [36] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [37] T. Lu, P. Neittaanmaki, and X.-C. Tai. A parallel splitting-up method for partial differential equations and its applications to Navier-Stokes equations. *ESAIM: Mathematical Modelling and Numerical Analysis*, 26(6):673–708, 1992.
- [38] Y. Lu, Z. Li, D. He, Z. Sun, B. Dong, T. Qin, L. Wang, and T.-y. Liu. Understanding and improving transformer from a multi-particle dynamic system point of view. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*.
- [39] S. Martin, A. Gagneux, P. Hagemann, and G. Steidl. PNP-FLOW: Plug-and-play image restoration with flow matching. In *International Conference on Learning Representations*, 2025.
- [40] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley. Deep learning for healthcare: review, opportunities and challenges. *Briefings in Bioinformatics*, 19(6):1236–1246, 2018.
- [41] Y. Quan, T. Zheng, and H. Ji. Pseudo-siamese blind-spot transformers for self-supervised real-world denoising. In *Advances in Neural Information Processing Systems*, volume 37, pages 13794–13817, 2024.
- [42] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [43] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-assisted Intervention—MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.
- [44] D. Ruiz-Balet and E. Zuazua. Neural ODE control for classification, approximation, and transport. *SIAM Review*, 65(3):735–773, 2023.
- [45] L. Ruthotto and E. Haber. Deep neural networks motivated by partial differential equations. *Journal of Mathematical Imaging and Vision*, 62(3):352–364, 2020.
- [46] Z. Shen, A. Havrilla, R. Lai, A. Cloninger, and W. Liao. Transformers for learning on noisy and task-level manifolds: Approximation and generalization insights. *arXiv preprint arXiv:2505.03205*, 2025.
- [47] R. Strudel, R. Garcia, I. Laptev, and C. Schmid. Segmenter: Transformer for semantic segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7262–7272, 2021.
- [48] X.-C. Tai, H. Liu, and R. Chan. PottsMGNet: A mathematical explanation of encoder-decoder based neural networks. *SIAM Journal on Imaging Sciences*, 17(1):540–594, 2024.
- [49] X.-C. Tai, H. Liu, R. H. Chan, and L. Li. A mathematical explanation of UNet. *Mathematical Foundations of Computing*, 2024.
- [50] S. Takakura and T. Suzuki. Approximation and estimation ability of transformers for sequence-to-sequence functions with infinite dimensional input. In *International Conference on Machine Learning*, pages 33416–33447. PMLR, 2023.

- [51] R. E. Turner. An introduction to transformers. *arXiv preprint arXiv:2304.10557*, 2023.
- [52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, volume 30, 2017.
- [53] T. Wang, Z. Dou, C. Bao, and Z. Shi. Diffusion mechanism in residual neural network: theory and applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(2):667–680, 2023.
- [54] E. Weinan. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5(1):1–11, 2017.
- [55] E. Weinan, C. Ma, and L. Wu. Machine learning from a continuous viewpoint, I. *Science China Mathematics*, 63(11):2233–2266, 2020.
- [56] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan, and L. Zhang. CvT: Introducing convolutions to vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22–31, 2021.
- [57] J. Xu, X. Sun, Z. Zhang, G. Zhao, and J. Lin. Understanding and improving layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [58] Y. Yang, J. Sun, H. Li, and Z. Xu. ADMM-CSNet: A deep learning approach for image compressive sensing. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(3):521–538, 2018.
- [59] D. Yarotsky. Error bounds for approximations with deep ReLU networks. *Neural Networks*, 94:103–114, 2017.
- [60] Z. Ye, X. Huang, L. Chen, H. Liu, Z. Wang, and B. Dong. PDEformer: Towards a foundation model for one-dimensional partial differential equations. In *ICLR 2024 Workshop on AI4DifferentialEquations In Science*, 2024.
- [61] T. Young, D. Hazarika, S. Poria, and E. Cambria. Recent trends in deep learning based natural language processing. *IEEE Computational Intelligence Magazine*, 13(3):55–75, 2018.
- [62] S. Yun, M. Jeong, R. Kim, J. Kang, and H. J. Kim. Graph transformer networks. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [63] B. Zhang and R. Sennrich. A lightweight recurrent network for sequence modeling. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1538–1548, 2019.
- [64] D.-X. Zhou. Universality of deep convolutional neural networks. *Applied and Computational Harmonic Analysis*, 48(2):787–794, 2020.
- [65] J. Zhu, X. Chen, K. He, Y. LeCun, and Z. Liu. Transformers without normalization. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14901–14911, 2025.
- [66] A. Ziaee and E. Çano. Batch layer normalization a new normalization layer for CNNs and RNNs. In *Proceedings of the 6th International Conference on Advances in Artificial Intelligence*, pages 40–49, 2022.